

CÉGEP De Chicoutimi

POST-MORTEM

MINI PROJET #2 - Dév. App. Avancé | 420-E80-CH-A26 | Gr. 01

Henri Pitre, Mathieu Tenret, Étienne Tremblay, Thomas
Derdeyn
2026-10-09

TABLE DES MATIÈRES

TABLE DES MATIÈRES	2
1. INFORMATIONS ESSENTIELLES	3
1.1 Lien GIT	3
1.2 Liste des documents produits	4
2. CHANGEMENTS APPORTÉS LORS DE L'ARRIMAGE	5
2.1 Suppression de classes inutiles (Ex : Player).....	5
2.2 Uniformisation de MessageType et du contenu de notre objet Message dans le diagramme de séquence à partir du diagramme de classes	5
2.3 Ajustement des diagrammes de classe/séquence lors de l'ajout de nouvelles fonctionnalités.	5
3. MATRICE DE COMPATIBILITÉ	5
4. ÉLÉMENTS APPRIS ET RÉALISATIONS	6
4.1 Henri Pitre	6
4.1.1 ÉLÉMENTS APPRIS	6
4.1.2 RÉALISATIONS	6
4.2 Mathieu Tenret.....	7
4.2.1 ÉLÉMENTS APPRIS	7
4.2.2 RÉALISATIONS	7
4.3 Étienne Tremblay.....	7
4.3.1 ÉLÉMENTS APPRIS	7
4.4.2 RÉALISATIONS	8
4.4 Thomas Derdeyn	8
4.4.1 ÉLÉMENTS APPRIS	8
4.4.2 RÉALISATIONS	8

1. INFORMATIONS ESSENTIELLES

1.1 Lien GIT

Puisque les quatre projets ont été réalisés différemment, aucun Git partagé n'a été fait. En effet, afin d'éviter de copier l'un sur l'autre et de s'inspirer trop fortement d'un collègue, le partage du code a été minimisé, et nous nous sommes seulement partagés les exécutable hormis quelques exceptions de débogage.

Cependant, nous avons bien évidemment utilisé Git individuellement lors du projet;

Nom	Lien Git
Henri Pitre	https://github.com/HenriPitre2432978/A26_DAE-Pitre_2432978.git
Mathieu Tenret	https://github.com/cegep-chicoutimi/2633645_TENRET_MATHIEU_RobotBattle_vf.git
Étienne Tremblay	https://github.com/cegep-chicoutimi/420_E80_Robot_Fight_ET
Thomas Derdeyn	https://github.com/cegep-chicoutimi/A26_DAE-Derdeyn_2634156_RobotCombat

1.2 Liste des documents produits

Nom du document	Utilité
Document de planification de documents (Individuel)	Théoriser ce qu'un document de conception devrait contenir pour être complet.
Document de planification de documents (Équipe)	Associer nos connaissances de conceptions pour former une maquette document de conception convenable
ANALYSE, MODÉLISATION, DOCUMENTATION ET CONCEPTION	Regrouper tous les diagrammes, scénarios, échanciers et planifications au même endroit, et les expliquer clairement pour assurer une compréhension maximale lors de la production
Diagramme des uses cases	Cas d'utilisation global généralisant le déroulement d'une utilisation normale d'un utilisateur quelconque
Diagramme de classes métier	Sert de référence absolue pour uniformiser les quatre projets ensemble pour s'assurer qu'ils communiquent bien et ont les mêmes noms et constantes qui désignent les mêmes choses et les mêmes quantités.
Diagramme de séquence	Référence absolue seconde qui précise le fonctionnement exact du programme, pour s'assurer que les programmes implémentent toutes les mêmes fonctionnalités et ce au même moment.
Scénarios	Équivalent d'un cas d'utilisation sous forme de texte permettant de clarifier des situations ambiguës. Cas associés : • Choisir son rôle; • Configurer la connexion; • Créer une partie; • Rejoindre une partie; • Répartir ses points d'amélioration; • Jouer un tour; • Rejouer une partie; • Quitter une partie.
Échéancier	Permet de situer dans le temps la réalisation de chaque étape pour assurer de remettre le projet en bonne et due forme
Planification du plan de tests	Permet d'assurer le bon fonctionnement du projet à l'aide de la planification de certains tests qui devront être implémentés

2. CHANGEMENTS APPORTÉS LORS DE L'ARRIMAGE

2.1 Suppression de classes inutiles (Ex : Player)

Alors que dans la version initiale nous avions un Player qui contenait un Robot, nous avons réalisé que cela était plutôt inutile, et qu'un joueur pouvait simplement n'être qu'un Robot.

2.2 Uniformisation de *MessageType* et du contenu de notre objet *Message* dans le diagramme de séquence à partir du diagramme de classes

Grâce à l'uniformisation des messages émis entre les deux systèmes, nous avons pu commencer à communiquer entre nos solutions. Pour certaines personnes nous avons dû ajouter le concept de sérialisation

En alignant les deux, nous avons évité un évènement qui aurait pu nous faire perdre beaucoup de temps à la fin. Si le diagramme de classes était différent que celui de séquence, deux coéquipiers auraient pu avoir deux messages de communications différents, ce qui rendrait le transfert de données impossible. Heureusement, nous nous sommes rendus compte du problème rapidement.

2.3 Ajustement des diagrammes de classe/séquence lors de l'ajout de nouvelles fonctionnalités.

En s'assurant de remettre au goût du jour les diagrammes (Ajout de Random,Dodge,Repair,Escape...), nous avons pu développer les nouvelles fonctionnalités individuellement en ayant la confiance que nous pourrions communiquer entre nous par la suite.

3. MATRICE DE COMPATIBILITÉ

Client \ Serveur	Étienne	Mathieu	Henri	Thomas
Étienne	OK	OK	OK	OK
Mathieu	NON	OK	OK	OK
Henri	NON	OK	OK	OK
Thomas	NON	OK	OK	OK

4. ÉLÉMENTS APPRIS ET RÉALISATIONS

4.1 Henri Pitre

4.1.1 ÉLÉMENTS APPRIS

La conception de A à Z d'un programme qui utilise le protocole TCP. J'ai appris à définir, utiliser et bénéficier de messages TCP clairs pour s'assurer d'une bonne communication, et également à répartir adéquatement les tâches reliées au serveur et au client. (Calcul + Affichage serveur, Affichage client)

Production de documents PUMML plus complexes : Bien que Software Ideas Modeler est très intuitif et pratique, il rend la conception bien plus lente puisqu'on doit imaginer visuellement la conception plutôt que de l'associer. Ainsi, j'ai appris à maîtriser PlantUML pour réaliser des diagrammes de qualité «quasi-professionnelles » (le diagramme de séquence) qui ont pu permettre une meilleure compréhension du projet pour mes coéquipiers et moi-même.

J'ai compris (à la dure) l'importance de s'en tenir à la conception en tout temps, et de faire savoir les changements si applicables, au risque de s'en mordre les doigts plus tard. En levant le drapeau rouge rapidement et en annonçant le changement effectué, nous avons évité de nombreuses heures de débogage inutile liés à des changements personnels qui n'ont pas été partagés aux autres.

4.1.2 RÉALISATIONS

J'ai réalisé l'importance de tenir les documents à jour, et de s'y fier. En effet, lorsque que le projet était entamé, nous avons remarqué que le diagramme de séquence et celui de classes, nos deux diagrammes principaux, n'avaient pas exactement les mêmes noms et désignations. Ainsi, certains se fiaient à l'un alors que dans un second projet, il se fiait à l'autre. Cela donnait donc des noms de variables différents, ce qui aurait pu nous poser bien des problèmes.

J'ai réalisé qu'un programme utilisant une connexion inter-ordinateur était bien plus facile qu'il ne le paraissait. En effet, le fonctionnement en TCP est bien plus simple et me faisait plus peur qu'il ne l'était réellement : l'essentiel est d'avoir une connexion

stable, et une communication complètement uniforme. Tant que ces deux choses sont respectés, le reste est très simple.

4.2 Mathieu Tenret

4.2.1 ÉLÉMENTS APPRIS

Communication client-serveur avec des sockets TCP (utilisation de la balise EOM et échange de données entre un client et un serveur)

Gestion d'un protocole de communication complet (les différents types de messages ont dû être gérés afin de définir comment le serveur et le client devaient réagir)

4.2.2 RÉALISATIONS

J'ai compris l'importance de définir un protocole de communication clair entre différents membres d'une équipe pour éviter les erreurs au moment de fusionner les différentes parties. Avec du recul, nous aurions dû probablement approfondir notre analyse un peu plus loin (en précisant par exemple comment les messages devaient être délimités), pour éviter certaines divergences entre les codes, qui ont entraînés des erreurs lors de leur mise en commun

J'ai appris l'importance de l'utilisation d'outils de débogage (comme les Loggers, qui permettent de suivre l'exécution d'un programme et d'ainsi détecter plus facilement l'emplacement et la cause des erreurs)

4.3 Étienne Tremblay

4.3.1 ÉLÉMENTS APPRIS

L'usage des loggers et l'importance dans la communication tcp entre user et server.

Implémentation des Interfaces

4.4.2 RÉALISATIONS

La Communication Tcp est plus facile à réaliser si tu comprends la logique de l'ordre des sends et receive. Elle peut être dure si tu as pas compris la séquence du logiciel et des actions de chaque acteurs.

La gestion des exceptions et des validations sont deux choses à ne pas se mêler. Si tu ne comprends pas quand bien les appelés et où les implémenter. Le code sera plus confus.

4.4 Thomas Derdeyn

4.4.1 ÉLÉMENTS APPRIS

La conception et l'échange de d'un protocole de socket synchrone en C#. J'ai pris connaissance de comment connecter un serveur à un client, les exceptions rejetées ainsi que comment communiquer correctement entre ces machines (tag et sérialisation de message).

L'utilisation d'un "Logger" propre à C# afin de faciliter le débogage et le développement ("Serilog").

La création d'une architecture client-serveur au sein d'un même projet avec des permissions d'action différentes où le serveur gère la partie.

4.4.2 RÉALISATIONS

J'ai réalisé l'importance de suivre les éléments de conception et de partager directement en groupe les différents éléments problématiques. En effet avec une conception un peu différente, une solution ne pouvait pas exécuter toutes les actions attendues.

J'ai réalisé l'importance de bien gérer les exceptions et les cas limites afin d'éviter d'arrêter le programme.